De “Hola Usuario capybara” a Root: MySQL, un Hash Crackeado y un PDF Mal Censurado en Unrecover

Writeup de **Unrecover**, máquina de dificultad media de DockerLabs. La web filtra un nombre de usuario en su propio saludo, y ese usuario resulta tener la misma contraseña débil en **MySQL** — pero no en FTP ni en SSH. Dentro de la base de datos hay un segundo usuario con su hash, que se cracquea para entrar por SSH. Y el remate: un backup con la contraseña de **root** “censurada” con un pixelado que no censura nada.

Reconocimiento

Primer `ping` para confirmar el sistema operativo por el TTL:

```
ping 172.17.0.2 -c1
```

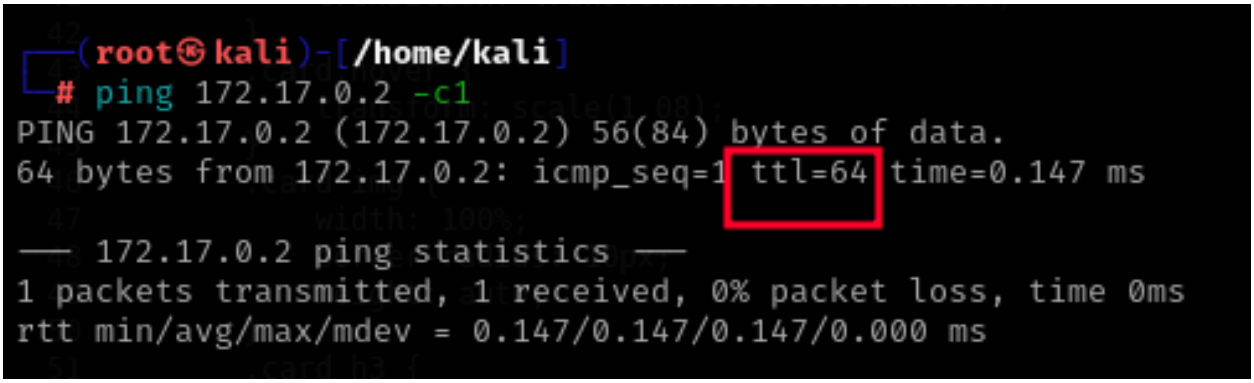


Figure 1: ping con ttl=64

`ttl=64` -> máquina Linux. Escaneo completo con Nmap:

```
sudo nmap -sS -p- -sC -sV -Pn 172.17.0.2
```



Figure 2: escaneo completo de nmap

Cuatro puertos abiertos, todos con versión identificada:

- **21/tcp** — vsftpd 3.0.3
- **22/tcp** — OpenSSH 9.2p1 Debian
- **80/tcp** — Apache httpd 2.4.62, título `Zoo de Capybaras`
- **3306/tcp** — MySQL, MariaDB 5.5.5-10.11.6

El 3306 expuesto ya es una pista en sí mismo: una base de datos no debería estar accesible desde fuera, y si lo está, es una vía más de ataque además de FTP y SSH.

La web filtra un usuario

```
curl http://172.17.0.2/
```

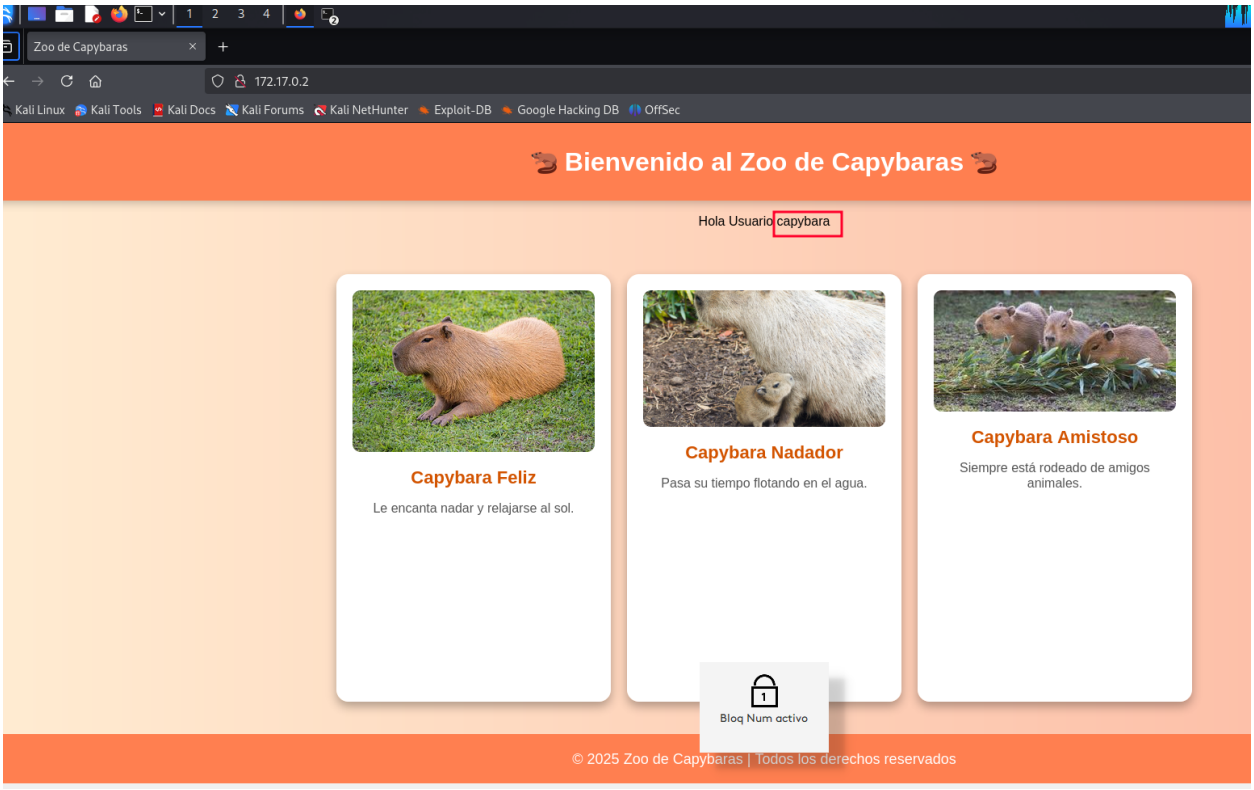


Figure 3: Zoo de Capybaras, saludo con el usuario capybara

La propia página saluda con `Hola Usuario capybara` — un nombre de usuario real, regalado sin pedir nada a cambio. Revisando el código fuente (`Ctrl+U`) no hay nada más interesante, así que el usuario `capybara` pasa a ser el dato fijo para atacar los tres servicios con usuario conocido.

Fuerza bruta contra FTP, SSH y MySQL

```
hydra -l capybara -P /usr/share/wordlists/rockyou.txt ftp://172.17.0.2
hydra -l capybara -P /usr/share/wordlists/rockyou.txt ssh://172.17.0.2
hydra -l capybara -P /usr/share/wordlists/rockyou.txt mysql://172.17.0.2
```

FTP y SSH no dan ningún resultado. MySQL sí: `capybara : password1`. La misma cuenta que no sirve para entrar por SSH o FTP, sí tenía una contraseña débil para la base de datos — un recordatorio de que cada servicio hay que probarlo por separado, nunca asumir que todos comparten (o no comparten) credenciales.

Enumerando la base de datos

```
mysql -u capybara -ppassword1 -h 172.17.0.2 --skip-ssl
```

(`--skip-ssl` para evitar el error de negociación TLS del cliente contra este MariaDB.)

```
show databases;
```

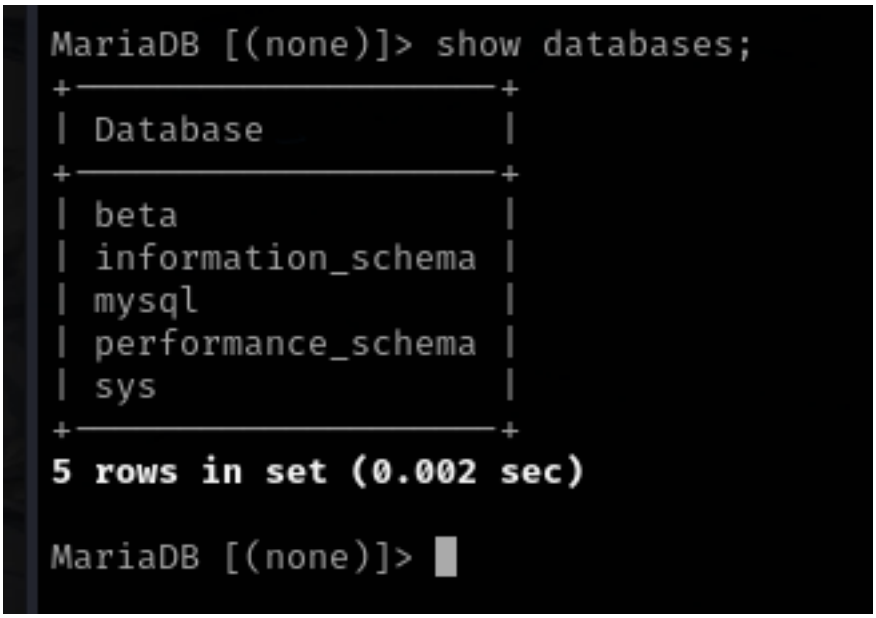


Figure 4: show databases en MariaDB

```
use beta;
show tables;
select * from registraton;
```

La tabla se llama `registraton` (no `registration` — el primer intento falla por el typo del propio desarrollador). Dentro, un segundo usuario:

- **Usuario:** `balulero`
- **Hash:** `520d3142a140addb8be7d858a7e29e15`

Patrón clásico: el acceso a una base de datos casi siempre da pie a leer su tabla de usuarios, y ahí suele haber credenciales de **otras** cuentas del sistema — a veces reutilizadas para SSH.

Crackeando el hash

```
hashcat -m 0 520d3142a140addb8be7d858a7e29e15 /usr/share/wordlists/rockyou.txt
```

En este caso se resolvió directamente contra CrackStation:

Resultado: `password1` — exactamente la misma contraseña que ya tenía `capybara` en MySQL. Dos usuarios distintos, la misma contraseña débil reutilizada.

Confirmando credenciales y entrando por SSH



Figure 5: tabla registraton con usuario balulero y su hash

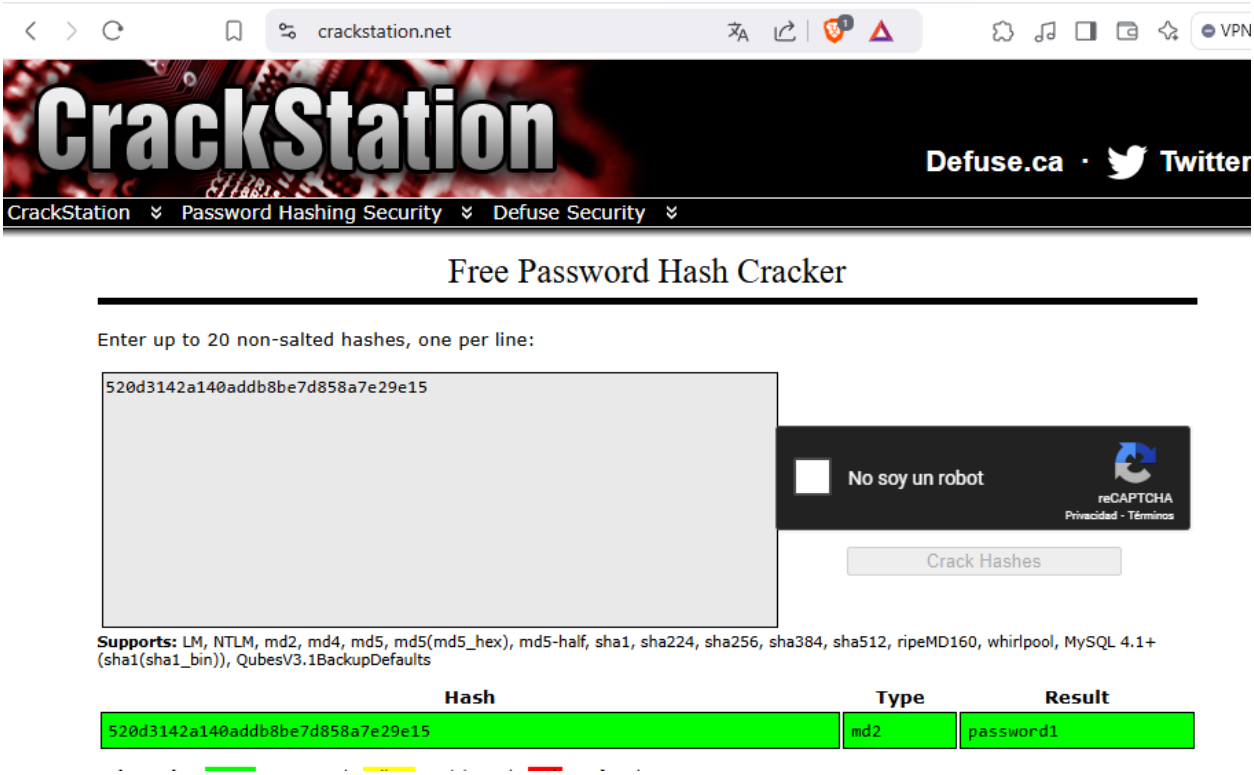


Figure 6: CrackStation identifica el hash y lo cracquea a password1

```
hydra -l balulero -P /usr/share/wordlists/rockyou.txt ftp://172.17.0.2
```

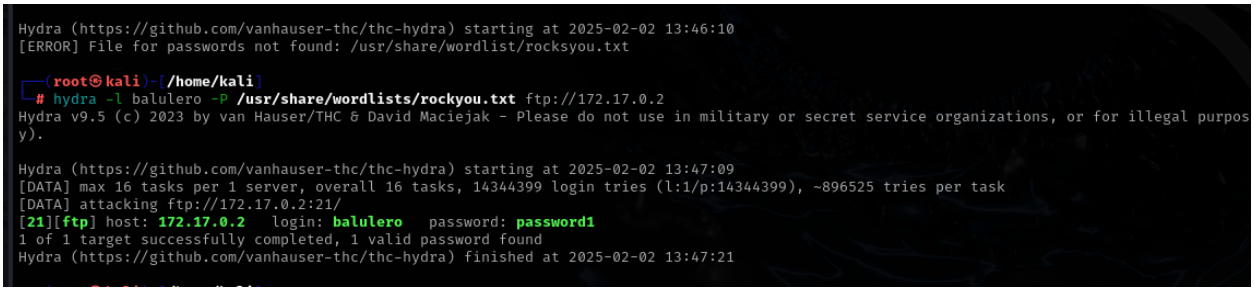


Figure 7: hydra confirmando password1 para balulero

Con la contraseña ya confirmada, acceso directo por SSH:

```
ssh balulero@172.17.0.2
```

Escalada de privilegios: primeros callejones sin salida

```
id; whoami
```

```
sudo -l
```

Ni `sudo` está instalado para este usuario. Revisando usuarios del sistema:

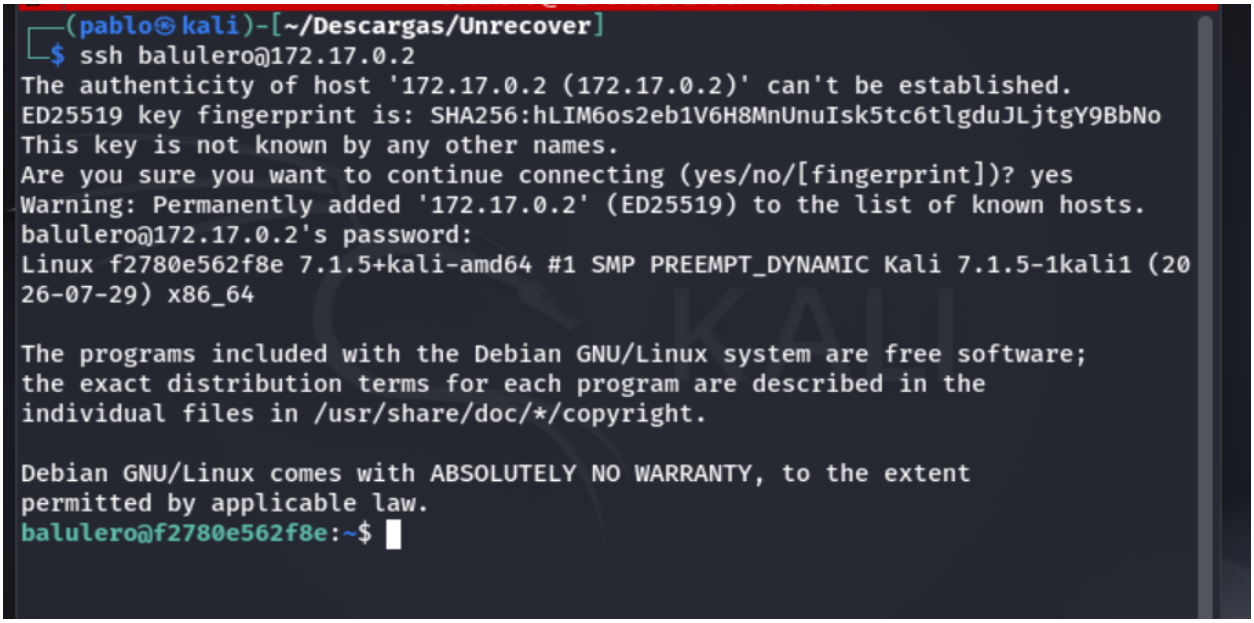


Figure 8: login SSH como balulero

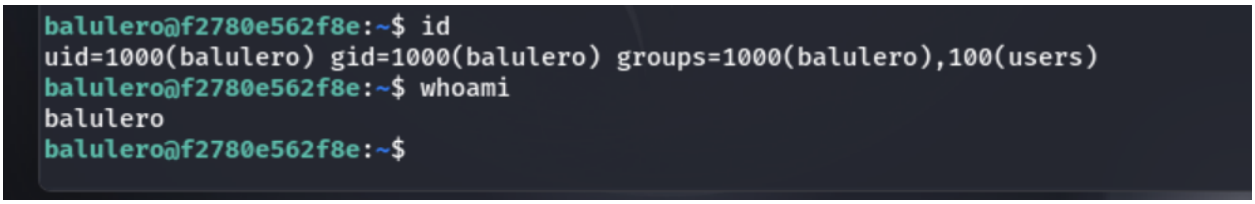


Figure 9: id y whoami como balulero

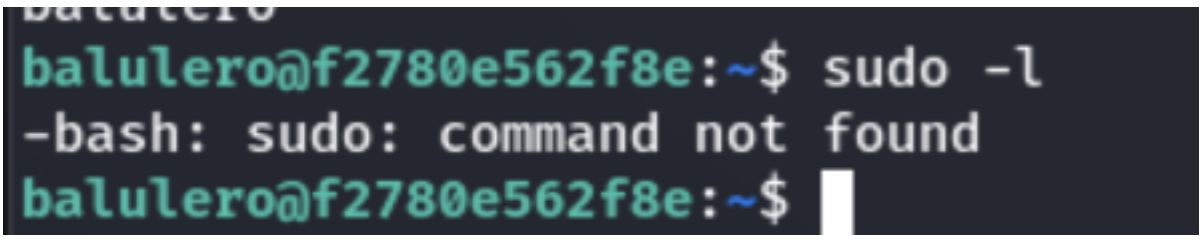


Figure 10: sudo -l sin sudo instalado

```
cat /etc/passwd
```

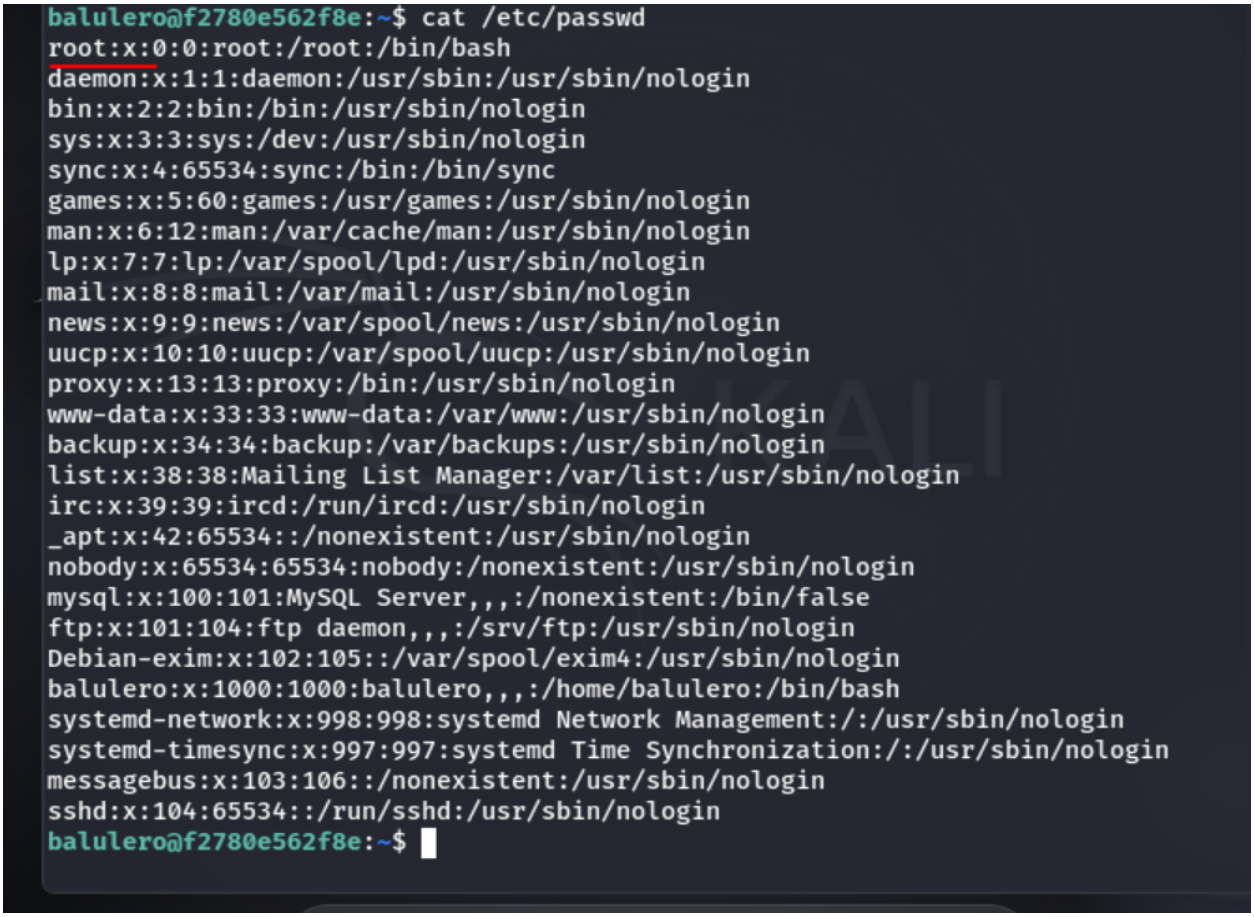


Figure 11: listado de /etc/passwd

El backup olvidado

```
pwd
ls
```

```
cd server/
ls -lah
```

Un `backup.pdf` propiedad de `root`, pero legible por cualquiera (`-rw-r--r--`).

Sacando el PDF de la máquina

Intento directo de copiarlo al directorio web, sin permisos:

```
cp backup.pdf /var/www/html/
```

Alternativa: servirlo con un servidor HTTP propio desde la misma carpeta:

```
python3 -m http.server
```

Y descargarlo desde el navegador apuntando a `IP:8000`:

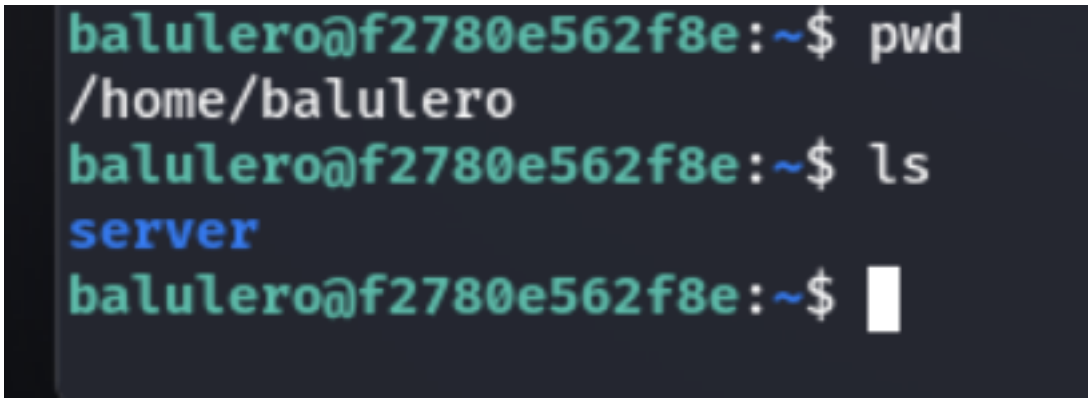


Figure 12: pwd y ls, aparece un directorio server

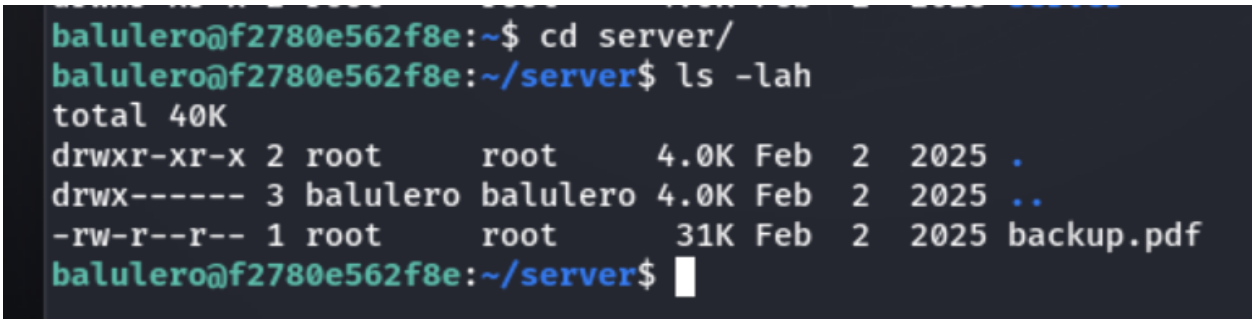


Figure 13: listado de server/ con backup.pdf propiedad de root

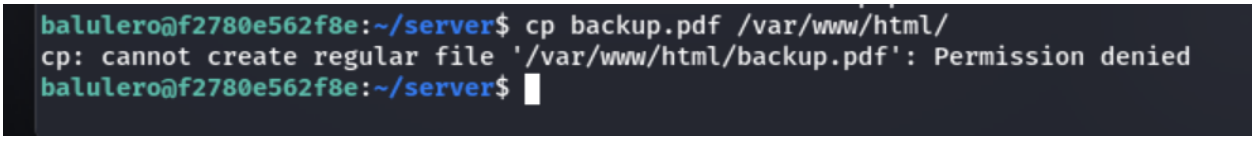


Figure 14: permiso denegado al copiar a /var/www/html

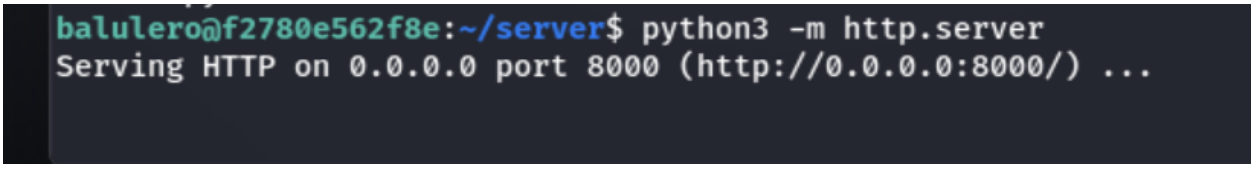


Figure 15: python3 -m http.server sirviendo en el puerto 8000

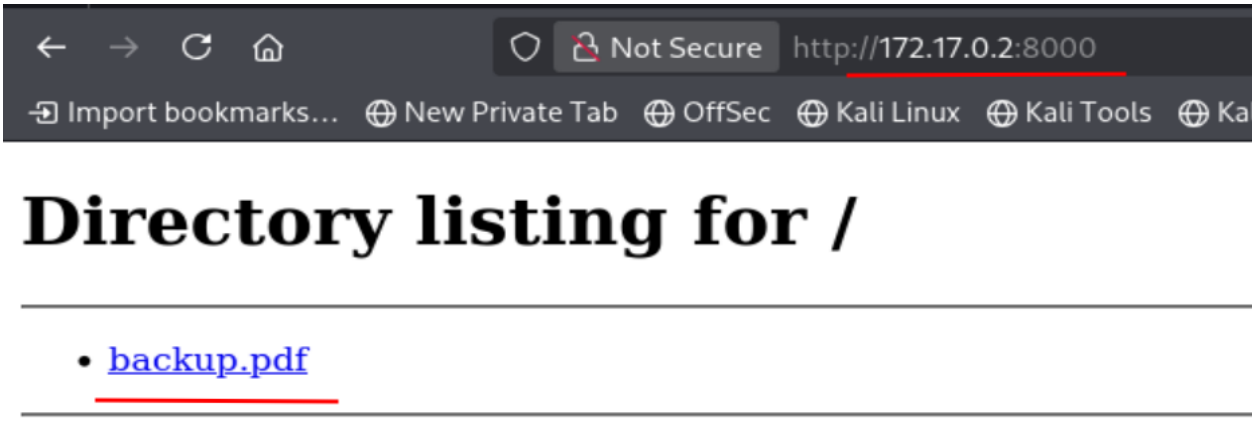


Figure 16: directory listing con backup.pdf descargable

Una censura que no censura nada

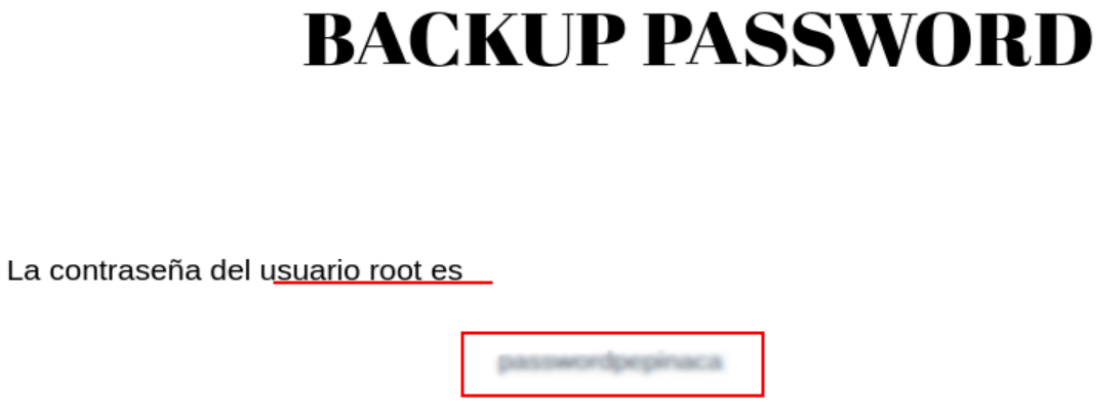


Figure 17: backup.pdf con la contraseña de root pixelada pero legible

El PDF indica la contraseña de `root`, pero aparece "difuminada" con un pixelado — que resulta perfectamente legible: `passwordpepinaca`. Difuminar o pixelar texto en un documento no es una forma segura de redactarlo: si el dato sigue ahí (o el pixelado es suave, como aquí), es recuperable sin ninguna herramienta especial, a simple vista.

Root

```
ssh root@172.17.0.2
```

`passwordpepinaca` era, en efecto, la contraseña de `root`. Laboratorio completado.

Lo que me llevo de esta máquina

- **Un saludo de bienvenida ("Hola, `<usuario>`") puede ser tan peligroso como un listado de directorios.** La web regaló el usuario `capybara` sin que nadie lo pidiera, y ese fue el único dato que hizo

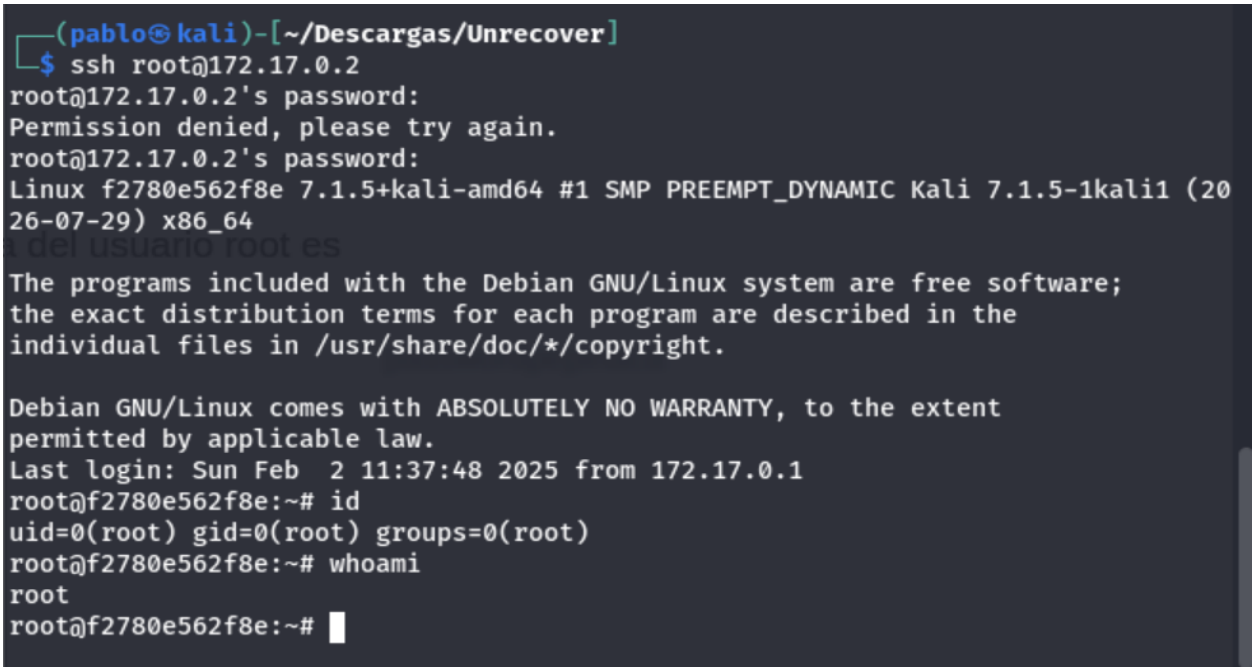


Figure 18: acceso como root, id y whoami confirmando

falta para arrancar toda la cadena.

- **La fuerza bruta no es "todo o nada" contra una máquina**: `capybara` falló en FTP y SSH, pero tenía una contraseña débil justo en MySQL. Hay que probar el mismo usuario contra **todos** los servicios expuestos, no descartar el ataque por un primer fallo.
- **Password reuse entre cuentas distintas** (`capybara` y `balulero` compartían `password1`) convierte el compromiso de una sola cuenta débil en el compromiso de varias.
- **Pixelar o difuminar texto en un PDF no es redactarlo.** Si el dato original sigue en el archivo o el efecto visual no destruye la información, se lee perfectamente — aquí sin necesidad de ninguna herramienta de "des-pixelado", a simple vista.
- **Un archivo "solo lectura" (`-rw-r--r--`) sigue siendo una fuga** si su contenido es sensible — los permisos de escritura no son lo único que importa a la hora de proteger un archivo.

Relacionadas

- [[Fuerza Bruta con Hydra]]
- [[Metasploit Framework]]
- [[Aidor - De IDOR a Root]]